

1. Perform the following operations and answer the associated questions (7 points):

a. Access the man pages, what is the difference between stat(1) and stat(2)?

The difference between stat(1) and stat(2) is that stat(1) is a shell command we would use in our terminal to return file information. Stat(2) is a C system call that we would use within our C program to retrieve information about a file through code.

b. Compile and test sampleProgramOne. Run it twice: use the sampleProgramOne source code file and then its executable file as test inputs. What exactly does sampleProgramOne do?

Sample program one prints out the mode of the file, and that it's a regular file. So for the source code, sampleProgramOne.c it tells us that this has read/write permissions for me and read for others, and for the compiled code, it tells us that it's an executable. Additionally, it tells us the inode number of the passed file.

c. Modify sampleProgramOne so that it reports whether a file is a directory or not. Verify that your program works (as shown below). Include a screenshot of the execution. Also include your source code as an attachment when uploading to blackboard.

```
((base) Alecs-MacBook-Pro-2:lab12 alecmirambeau$ ./SampleProgramOne ./sampleProgramOne.c
value is: 33188
inode value is: 190556307
The file is NOT a directory.
(base) Alecs-MacBook-Pro-2:lab12 alecmirambeau$ stat sampleProgramOne.c
16777232 190556307 -rw-r--r-- 1 alecmirambeau staff 0 919 "Apr  2 10:51:52 2025" "Apr  2 10:51:
51 2025" "Apr  2 10:51:51 2025" "Apr  2 09:56:12 2025" 4096 8 0 sampleProgramOne.c
(base) Alecs-MacBook-Pro-2:lab12 alecmirambeau$ █
```

We can see that my compiled sampleProgramOne.c prints out that sampleProgramOne.c is NOT a directory, and when we use the stat command with sampleProgramOne.c, it puts out "-rw-r--r-", notice how it doesn't start with a d. That is because it's not a directory, otherwise it would start with a d instead of a -. So my code and the stat command are in agreement.

2. Perform the following operations and answer the associated questions (9 points):

a. Compile and test sampleProgramTwo, what exactly does sampleProgramTwo do?

SampleProgramTwo makes a directory stream from the current directory you're executing the compiled code from, and prints out all the files in this directory, and then closes this directory stream.

b. Modify sampleProgramTwo so that it also reports the size of each file (in bytes). Ensure your output is human-friendly (i.e., readable). Verify and demonstrate the correctness of your program by testing it against the ls program using your current directory. For example 'ls -l' should return the same value as './sampleProgramTwo'. Submit your modified program and include a screenshot of the execution.

```

[(base) Alecs-MacBook-Pro-2:lab12 alecmirambeau$ ls -la
total 160
drwxr-xr-x@  9 alecmirambeau  staff    288 Apr  2 12:03 .
drwxr-xr-x@ 19 alecmirambeau  staff    608 Apr  2 09:56 ..
-rwxr-xr-x@  1 alecmirambeau  staff  33856 Apr  2 10:51 SampleProgramOne
drwxr-xr-x@  3 alecmirambeau  staff     96 Apr  2 10:11 SampleProgramOne.dSYM
-rwxr-xr-x@  1 alecmirambeau  staff  33920 Apr  2 12:03 SampleProgramTwo
drwxr-xr-x@  3 alecmirambeau  staff     96 Apr  2 11:28 SampleProgramTwo.dSYM
-rw-r--r--@  1 alecmirambeau  staff    919 Apr  2 10:51 sampleProgramOne.c
-rw-r--r--@  1 alecmirambeau  staff    425 Apr  2 12:02 sampleProgramTwo.c
-rw-r--r--@  1 alecmirambeau  staff     0 Apr  2 12:00 test.c
[(base) Alecs-MacBook-Pro-2:lab12 alecmirambeau$ ./SampleProgramTwo
.                size: 288 Bytes
..               size: 608 Bytes
sampleProgramOne.c  size: 919 Bytes
SampleProgramTwo.dSYM size: 96 Bytes
SampleProgramTwo    size: 33920 Bytes
SampleProgramOne    size: 33856 Bytes
test.c              size: 0 Bytes
SampleProgramOne.dSYM size: 96 Bytes
sampleProgramTwo.c  size: 425 Bytes
(base) Alecs-MacBook-Pro-2:lab12 alecmirambeau$ █

```

3. Answer the following questions (9 points):

- a. Use `du` to report the usage of all the files in some of your directories (be sure to choose some with subdirectories), based on the order of information provided, which of the two tree traversal algorithms does `du` use?

Depth first search

- b. What is the default block size used by `du`?

blocksize is rounded up to the next multiple of 512

- c. Speculate: given the intended purpose of `du`, why is the usage reported in blocks, instead of bytes?

Most files nowadays aren't 512 bytes small, so using a block size allows us to do a simple calculation of $\# \text{ of blocks} \times 512 = \text{total number of bytes}$. Plus it would be less readable to print out the exact number of bits or bytes as programs nowadays are more complex and bigger and will continue to be this way. Plus it may have something to do with early hdds being really small vs. now 512 GB is the standard.

Programming Assignment

```
● (base) Alecs-MacBook-Pro-2:lab12 alecmirambeau$ ./run .
specified directory is: .
././ The user ID is: 501, the group ID is: 20. The inode value is: 190556290
./.. The user ID is: 501, the group ID is: 20. The inode value is: 186056592
./sampleProgramOne.c The user ID is: 501, the group ID is: 20. The inode value is: 190556307
./run.dSYM The user ID is: 501, the group ID is: 20. The inode value is: 190575278
./SampleProgramTwo The user ID is: 501, the group ID is: 20. The inode value is: 190568258
./SampleProgramOne The user ID is: 501, the group ID is: 20. The inode value is: 190561432
./programAssignment.c The user ID is: 501, the group ID is: 20. The inode value is: 190574966
./run The user ID is: 501, the group ID is: 20. The inode value is: 190582190
./sampleProgramTwo.c The user ID is: 501, the group ID is: 20. The inode value is: 190564199
```

```
● (base) Alecs-MacBook-Pro-2:lab12 alecmirambeau$ ./run ../lab4
specified directory is: ../lab4
../lab4/. The user ID is: 501, the group ID is: 20. The inode value is: 186598068
../lab4/.. The user ID is: 501, the group ID is: 20. The inode value is: 186056592
../lab4/sampleProgramOne.c The user ID is: 501, the group ID is: 20. The inode value is: 186598158
../lab4/CompiledProgramAssignment The user ID is: 501, the group ID is: 20. The inode value is: 187379816
../lab4/test The user ID is: 501, the group ID is: 20. The inode value is: 186765473
../lab4/programAssignment.c The user ID is: 501, the group ID is: 20. The inode value is: 187379817
../lab4/sampleProgram2.c The user ID is: 501, the group ID is: 20. The inode value is: 186602013
../lab4/inputFile.txt The user ID is: 501, the group ID is: 20. The inode value is: 187367576
../lab4/CompiledSampleProgramThree The user ID is: 501, the group ID is: 20. The inode value is: 186605554
../lab4/CompiledSampleProgramTwo The user ID is: 501, the group ID is: 20. The inode value is: 186604107
../lab4/sampleProgramThree.c The user ID is: 501, the group ID is: 20. The inode value is: 186605272
../lab4/CompiledSampleProgramOne The user ID is: 501, the group ID is: 20. The inode value is: 186601027
../lab4/CompiledProgramAssignment.dSYM The user ID is: 501, the group ID is: 20. The inode value is: 187367634
../lab4/test.dSYM The user ID is: 501, the group ID is: 20. The inode value is: 186756115
```